

DESIGN_PRINCIPLES.md

Architectural principles for the `oil_network` model. Read alongside `CLAUDE.md` (full context) and `PROJECT_STATE.md` (current numbers).

The framework rests on a small set of **axioms** — design commitments that cannot be derived from anything else. Most of the other rules that earlier drafts treated as principles are actually **corollaries**: properties that fall out once the axioms are accepted and the variables collection is properly populated. The reorganisation makes clear what the framework genuinely enforces versus what it gets for free.

The single most load-bearing axiom is **Axiom 3 (variables are the source of truth)**. Once every structural fact about the graph is encoded in a variable or its `formula_inputs`, the rest of the framework — edges, hierarchy, node status, partition closure — is a view.

Axioms

The six independent commitments below cannot be derived from the others. They are the framework's primitive vocabulary.

Axiom 1 — Asset-centric representation

Every physical asset is a node with its own inventory and mass balance: pipelines, vessels, refineries, terminals, gathering systems, storage hubs. The alternative (location-centric, with pipelines as edge attributes) is explicitly rejected because in-transit volume — pipeline line-fill, vessel cargo, Jones Act tankers — is substantial in crude logistics and must be observable.

Example in the live model. The 2026-05-15 (twenty-second pass) refactor placed PADD-5's Jones Act in-transit residual (~6 MMbbl) onto the existing `inter_padd_3_to_5_agg` pipeline node as its inventory, closing PADD-5's stock partition exactly. A location-centric model would have had nowhere to put those barrels.

Axiom 2 — Stable topology via zero-flow edges

The adjacency matrix is fixed across time. Edges and nodes persist whether or not they carry flow in the current scenario; inactive edges have value zero, not absent. This is a precondition for standard GNN architectures, which assume the graph structure is constant per snapshot.

Example. `pipe_bakken_xstate` carries four flow variables (two inflows, two outflows) that are all `latent()` today, but the node and edges exist in every scenario. If per-operator gathering data eventually arrives, the values populate without any topological change.

Axiom 3 — Variables are the single source of truth

The variables collection plus the per-scenario `variable_assignments` rows fully determine every structural fact about the graph. Edges, partition links, aggregation roll-ups, and node status are all views over this single source.

Concretely: five layered materialised views fall out of the variables collection alone.

View	Reads from	What it derives
<code>v_flow_edges</code>	<code>variables</code> (relational rows)	Flow topology

v_aggregation_edges	formula_inputs on every assignment	Aggregation graph (any reference)
v_partition_tree	v_aggregation_edges filtered to same-type / same-related-node	The partition spine
v_node_status	variable_assignments	authoritative / derived / collapsed per (scenario, node)
v_node_pcisob	scenario_resolved_values	Per-node P/C/I/O/B/S/ΔS aggregates

No structural fact is stored twice. If the variables collection is consistent, every view is consistent. Renderers read these views exclusively — no hardcoded structural overrides anywhere.

Multi-parent rendering is a natural consequence. A node with multiple partition parents (e.g. pipe_bakken_xstate belongs to both PADD 2 and PADD 4 via its inventory-membership in padd{2,4}_view) appears under every parent. The JS's intra/boundary filter prevents double-counting because each edge is on the boundary of at most one parent context.

Axiom 4 — Persistent asset graph vs scenario state

Two layers, cleanly separated. The **persistent asset graph** is the fixed superset of every node and feasible edge that could ever matter. It does not change when data granularity improves. The **scenario state** is derived at load time from the asset graph plus per-scenario variable_assignments.

Data upgrades update variable_assignments; they never force a structural change to the asset graph. The earlier "coverage contract" concept turned out to be mechanically just the assignments table tagged with a scenario_id — no separate object is needed.

Example. When STEO basin-level data was added, no asset was created; only the existing permian.production etc. got TS bindings via new variable_assignments rows.

Axiom 5 — Observed XOR derived, enforced at the schema level

For each (scenario, variable, observation_date), exactly one of two states holds: **observed** (timeseries_id set) or **derived** (formula set). The schema enforces this with a CHECK constraint:

```
num_nonnulls(timeseries_id, formula) = 1
```

The constraint cannot be violated by INSERT — it is structural, not a post-hoc validator. The TS-binding uniqueness audit extends this to the TS attribution level: one TS, one variable per scenario.

The formula_inputs field carries a dual role depending on which side of the invariant the variable sits on. When the variable is observed (TS-bound), formula_inputs is the **constraint set** — v_aggregation_consistency checks $|TS_value - \Sigma(\text{constituents})|$. When the variable is derived (formula-bound), formula_inputs is the **operand set** — the formula resolves over these inputs. Same field, two roles, decided by context.

Example of the dual role. padd2_view.inventory is TS-bound to EIA's MCRSTP21, with formula_inputs = [padd2_tank_farms_pipelines, padd2_refinery_stocks]. The TS gives the value (~104 MMbbl at 2024-12-01); the formula_inputs trigger the consistency check that Σ -of-children equals the published value. Both sides closed to gap = 0 at 2024-12-01.

Axiom 6 — Bidirectional flows are two directed edges

For reversible pipelines and bidirectional terminals (Seaway, Capline, LOOP, SPR sites), each direction is a separate directed edge with its own flow variable. The zero-flow convention handles whichever direction is inactive in the current period. No single bidirectional edge anywhere in the model.

Example. `inter_padd_2_to_4_agg` and `inter_padd_4_to_2_agg` are separate aggregates; on a given month the active direction carries the EIA `MCRMP*` value and the other carries zero.

Corollaries

The properties below are consequences of the axioms plus a correctly-populated variables collection. They are useful to state for clarity but do not introduce new primitives.

Corollary A — Mass balance at every physical node; aggregates inherit it

Physical nodes satisfy

$$\Delta S(i, t) = P(i, t) + F_{in}(i, t) - C(i, t) - F_{out}(i, t) + B(i, t)$$

where `B` is the balancing item carrying systematic reporting discrepancies (IEA convention). This is a fact about the world for assets that actually move barrels.

Abstract / observational aggregates (PADDs, basin roll-ups, refining districts) do not introduce a *new* mass balance constraint. Their `P`, `C`, `I`, `O`, `B`, `S` variables are either roll-ups (Axiom 3: `formula = sum(children)`) or independently TS-observed at the aggregate level (e.g. EIA publishes USA-B). When both are available, the comparison is a **cross-check**, not a new constraint.

Example. At 2024-12-01, USA-B observed via `MCRUA_NUS_2` = -379 kbd; Σ -of-five-PADD-B = -380 kbd. Gap = 1 kbd. This isn't mass balance "holding at USA" in any new sense — it's the children summing within 1 kbd of the independent USA observation.

Corollary B — The observational aggregation layer has no edges

Falls out of Axiom 3: an "edge" is a relational variable (`inflow` or `outflow` with `related_node_id` set). Observational aggregates are formula-defined views — they reference physical nodes through `formula_inputs`, not through relational variables. Therefore they don't appear in `v_flow_edges`.

`v_partition_tree` further distinguishes the **partition spine** (same-type, same-related-node aggregation, with arithmetic-residual and bidirectional-cycle filters) from the broader `v_aggregation_edges`. Multiple aggregation views (PADD-based, basin-based, future operator-based) can coexist over the same physical node set without polluting the flow graph.

Corollary C — Labels are properties; hierarchies are formulas

Geography metadata (`lat`, `lon`, `state`, `county`, `PADD`, `country`) is a labelling scheme: every node carries labels that many other nodes may share. Labels describe properties.

Resolution hierarchies (Permian basin $\leftrightarrow$ Permian-TX $\leftrightarrow$ Permian-Midland-County) are aggregation relationships: declared by `formula_inputs` referencing children at finer granularity. A second hierarchy by operator (Shell, Chevron) or by grade (WTI, ASCI) could be declared the same way over the same physical nodes — multiple aggregations coexist as long as each is internally consistent (no double-counting within one tree).

Anti-pattern: rolling up by a label without checking resolution level. "All Texas-state production" would double-count if it included Permian-TX (basin level, sub-state) AND state-conventional-TX (state level, sub-Permian). Choose one resolution per aggregation.

The `assets.attributes->resolution_hierarchy` JSONB is redundant with the variables' `formula_inputs` graph and could be retired — most renderers already read `v_partition_tree` rather than the JSONB.

Corollary D — Latent allocation at junctions

Where flows split across multiple downstream routes and the per-route split is not observable in public data, the variable carries `formula = 'latent()'`. The resolver writes `value = NULL, source = 'latent'`. Mass balance still holds because pass-through node types (gathering, pipeline, import_terminal, export_terminal, foreign_export_destination, foreign_production_aggregate) default to $P = C = \Delta S = B = 0$, which forces $\Sigma F_{in} = \Sigma F_{out}$ at the junction even when individual edges are latent. The `v_node_balance_check` view materialises `sum_out_implied` so the constraint is queryable.

This is not a separate principle — it is `latent()` being one of the formula values the resolver dispatches on (see resolution rule canon below), combined with node-type defaults. Methodologically it commits to **declaring unknowns explicitly rather than guessing**, which is the position the thesis takes against ad-hoc rule-based allocation.

Example. Permian outflow splits across Gray Oak, Cactus II, Midland-to-ECHO, and others; per-route shares aren't publicly published. Each `pipe_*.inflow ← permian_tx_gathering` variable carries `latent()`. Mass balance constrains the un-observed flows to sum to the gathering node's total outflow (which is observed as the basin's production minus on-basin consumption). Future per-operator data lands on those variables without any structural change.

Corollary D-bis — Last-observation-carried-forward (LOCF) for TS-bound variables

A monthly EIA value represents either the average daily rate over the calendar month (bpd quantities) or the end-of-period stock (mbbl quantities). The framework's temporal convention is that the value applies to every subsequent date until a new observation lands — a step function between observations, no interpolation. This is implemented by `eval_observed` in the resolver: at each date, the most recent TS value at or before that date is the resolved value. Dates before the first observation get no row.

Audit traceability: fresh observations have `formula_used = NULL`; carried-forward rows record the source date as `formula_used = 'locf(YYYY-MM-DD)'`, so downstream consumers can distinguish "freshly published this month" from "carried over from N months ago", and a frequency-gap audit can flag long LOCF runs against the expected publication cadence.

When LOCF surfaces inconsistencies. If two sources with different temporal horizons feed into the same residual or balance equation, the framework can produce values that look surprising — typically negative quantities where physics would forbid them. The clearest case in the live model: `montana_other.production = montana_state_view.production - bakken_mt.production`. The state-level EIA series (MCRFPMT2) ends earlier than the STEO basin forecast (COPRBK); LOCF holds Montana flat while Bakken-MT continues forecasting upward, and the residual goes negative for the months between the state-data horizon and the STEO horizon. This is not a bug in the resolver — it is the framework being honest about a real divergence between source publications. The recommended response is a **frequency-gap check** that flags LOCF runs exceeding the expected source cadence; that check is future work.

Corollary E — Node status (authoritative / derived / collapsed) is a view

A node's status follows mechanically from its variables' assignments under the active scenario:

- **authoritative** — at least one variable on the node is TS-bound.
- **derived** — no TS, but at least one variable resolves to a non-zero value via formula.
- **collapsed** — every variable on the node is `'0'` or `'latent()'`.

Materialised by `v_node_status`. The earlier `nodes.starter_status` column was retired in the twelfth pass.

Resolution rule canon

The resolver dispatches every variable to exactly one of these rules (priority order top to bottom):

Rule	Trigger	Operation
observed	<code>timeseries_id</code> $\neq$ NULL	TS lookup
zero	<code>formula</code> = '0'	value = 0
latent	<code>formula</code> = 'latent()' (with reverse-mirror promotion when paired side resolves)	value = NULL
sum	<code>formula</code> = 'sum'	Σ over <code>formula_inputs</code>
alias	<code>formula</code> = bare <code>variable_id</code>	inherit value from that variable
arithmetic	<code>formula</code> = signed combination of <code>variable_ids</code> (e.g. <code>A - B - C</code>)	evaluate combination
closure	B variable with inputs spanning inventory/inflow/outflow	$B = \Delta S - P + C - \Sigma I + \Sigma O$
reverse-mirror	relational variable with no own resolution; paired side resolved	inherit paired value
partial	arithmetic input is NULL; resolver writes explicit (<code>value=NULL</code> , <code>source='partial'</code>) row	preserves the "tried and couldn't" record
unresolved	none of the above	should be zero in a healthy run

The **semantic role of a sum** (aggregation parent, fan-out total, residual) is recoverable from the structure of `formula_inputs`, not from the formula text: same-type same-commodity inputs $\rightarrow$ aggregation; mixed-type inputs on the same node $\rightarrow$ fan-out; explicit signed inputs $\rightarrow$ residual. The twelfth pass collapsed three earlier sum-style labels (`sum_over_children`, `sum_over_outflows`, `sum_same_type`) into one canonical sum rule on this basis.

What is *not* a principle of this framework

The reorganisation makes clear what the framework **does not** treat as primitive:

- **Mass balance at observational aggregates** is not an independent constraint (Corollary A). It's an algebraic consequence of mass balance at physical nodes plus the aggregation formulas. Useful as a cross-check when the aggregate is also TS-observed; not a new fact about the world.
- **The resolution hierarchy as a separate object** is not stored — it's encoded in `formula_inputs` (Corollary C).
- **Node status as a column** is not stored — it's a view (Corollary E).
- **The "coverage contract" as a primary object** is not stored — it's mechanically the assignments table per scenario (Axiom 4).

- **Latent allocation as a separate rule** is not invoked — it's just one of the formula values dispatched by the resolver (Corollary D).

Style conventions

- British spelling — modelled, organised, behaviour, colour.
- Harvard (author-date) referencing.
- Prose over bullets in the thesis text.
- Avoid AI-signature patterns: tricolons, roadmap sentences ("we now proceed to discuss..."), self-describing methodology language.

Anti-patterns to avoid

- Treating edges as first-class declarations independent of variables. Edges are a view (`v_flow_edges`).
- Storing the resolution hierarchy as a separate primary structure when it is derivable from `formula_inputs` (Corollary C).
- Storing node status as a column when it is a view (Corollary E).
- Storing the same fact in JSONB and a typed column.
- Mixing physical and observational nodes in the same edge graph (Corollary B).
- Using post-hoc validation for the observed/derived invariant when schema-level enforcement is available (Axiom 5).
- Rolling up by a geographic label without first picking a resolution level (Corollary C).
- Rebuilding the asset graph structurally when data granularity improves — that is what per-scenario `variable_assignments` are for (Axiom 4).
- Multiple resolver labels for the same operation.
- Adding forecasting work to the current thesis scope — explicitly future work.